C 语言课程设计代码标准

一、命名规范

1. 通用原则

- 见名知意，禁止无意义命名（如`a`、`b`、`temp`、`x`），优先使用英文全称，避免拼音/拼音缩写混用
- 命名风格统一

2. 各元素命名规则

元素类型	命名风格	示例
变量/函数	小驼峰	`studentScore`
常量/宏定义	全大写+下划线	`#define MAX_STUDENT 50`
结构体/枚举	大驼峰	`typedef struct StudentInfo {} StudentInfo;`
文件/目录	小写+下划线	`student_manager.c` / `utils/`

二、代码格式规范

1. 基础格式

- 缩进：Tab
- 空行：函数之间空1行，逻辑块（如if/for前后）空1行，禁止连续空多行

2. 符号格式

- 运算符两侧加空格：`a = b + c;`（而非`a=b+c;`）
- 逗号后加空格：`addStudent(1, "张三", 90.5);`
- 大括号风格（统一使用）：

```

if (score >= 60) {
    printf("及格\n");
} else {
    printf("不及格\n");
}
```

- `if/for/while`即使只有一行代码，也必须加大括号（避免逻辑漏洞）

三、注释规范（老师重点看）

1. 文件头部注释

每个`.c`文件开头必须加，示例：

```

/*
 * 文件名：student_manager.c
 * 功能：实现学生信息的增删改查、排序、保存
 * 作者：XXX
 * 日期：2026-03-11
 * 课程设计：《C语言程序设计》学生成绩管理系统
 */
```

2. 函数注释

核心函数（对外调用）必须加注释，说明功能、参数、返回值：

```

/**
 * 功能：添加学生信息
 * 参数：id-学生学号，name-学生姓名，score-学生成绩
 * 返回值：0-添加成功，1-添加失败（超出最大数量）
 */
int addStudent(int id, char* name, float score) {
    // 函数实现
}
```

3. 其他注释

- 复杂逻辑块（如链表操作、文件读写）上方加单行注释
- 禁止注释掉无用代码（直接删除，无需保留）
- 注释语言：中文（课程设计要求），简洁不啰嗦（只说“为什么”，不说“做什么”）

四、函数与代码结构

1. 函数设计

- 一个函数只做一件事，长度控制在50行以内
- 函数参数不超过4个，参数过多时用结构体封装
- 所有函数必须先声明后使用（声明写在`.h`文件或`.c`文件开头）

2. main函数规范（高分模板）

main函数只做流程控制，不写具体业务逻辑：

```

#include "student_manager.h"

```

```c
int main() {
    // 初始化
    initStudentData();

    // 主菜单循环
    while (1) {
        printMenu(); // 打印菜单
        int choice;
        scanf("%d", &choice);

        // 功能选择
        switch (choice) {
            case 1: addStudentUI(); break;   // 新增学生（UI层）
            case 2: delStudentUI(); break;   // 删除学生（UI层）
            case 3: queryStudentUI(); break; // 查询学生（UI层）
            case 0: saveStudentData(); return 0; // 保存数据并退出
            default: printf("输入错误，请重新选择！\n");
        }
    }
}
```

五、数据与安全规范

1. 数据类型与变量

- 变量就近定义（不用堆在函数开头）
- 尽量避免全局变量，必须使用时加注释说明用途
- 字符数组指定长度：`char name[20];`（避免缓冲区溢出）

2. 安全健壮性

- 数组/链表访问前检查边界：`if (index >= 0 && index < MAX_STUDENT)`
- 文件操作必判是否成功：

```c

FILE* fp = fopen("student.txt", "r");
if (fp == NULL) {
    printf("文件打开失败！\n");
    return 1;
}
```

- 输入合法性校验：`scanf`后判断输入是否符合预期（如数字/字符）
- 资源释放：文件使用后`fclose(fp)`，动态内存`free()`（配对使用）

六、文件组织规范

1. 文件拆分（课程设计标准结构）

文件名称	作用
main.c	主函数、菜单循环、流程控制
xxx_manager.h	结构体、常量、函数声明
xxx_manager.c	核心功能实现（增删改查等）

2. 头文件防护

`.h` 文件必须加防护，避免重复包含：

```

#pragma once  // 简洁版（推荐）
// 兼容版（备选）
#ifndef STUDENT_MANAGER_H
#define STUDENT_MANAGER_H

// 头文件内容（结构体、函数声明等）

#endif
```

3. 头文件引入顺序

```

// 1. 系统头文件
#include <stdio.h>
#include <stdlib.h>
// 2. 项目内部头文件
#include "student_manager.h"
```

七、提交与工程规范

1. 代码无警告：编译时开启所有警告（如 `-Wall`），确保零警告
2. 文件名规范：避免中文/特殊字符，如 `学生管理系统.c` → `student_manager.c`
3. 提交内容：只提交 `.c` / `.h` / `README.md`（可选），不提交编译生成的 `.exe` / `.o` 文件

总结

1. 命名、格式、注释是课程设计评分核心，务必统一风格、见名知意；
2. main函数只做流程控制，业务逻辑拆分到独立函数，降低耦合；
3. 重点关注数组越界、文件操作失败、输入非法等安全问题，提升代码健壮性。